

Understanding Variables, Data Types, and Naming Conventions



CONTENT

01

Physical Activity

02

Variables

03

Naming Conventions

04

Data Types





01

Physical Activity

Activity Name: "You are Avengers"



Iron man with
var Power



Spider man
with **let** Power



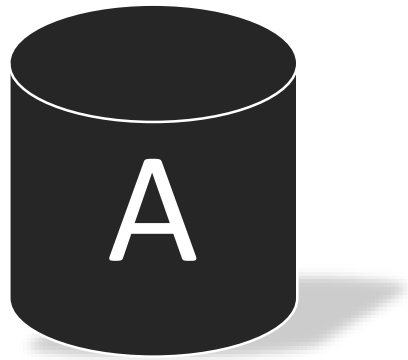
Hulk with **const**
Power



02

Variables

What is Variables?



container



```
let container = 'A'
```

a **variable** is a container used to store **data values**

How to declare variables?

You **declare** a variable using var, let, or const

Example :

var x = 5

let x = 5

const x = 5

var x = 'a'

let x = 'a'

const x = 'a'

var x = 'quad'

let x = 'quad'

const x = 'quad'

Note : var ,let or const is keyword , x is identifier , = is assignment operator
And 5 is value.



03

Naming Conventions



JavaScript Variable Naming: Dos, Don'ts, and Pro Tips

1. Use Camel Case for Variable Names

Camel Case: First word lowercase, next words capitalized.

Exp : `let userName = "quad"`

2. Never Start with a Number

Exp : `let 1name = "Wrong"` ❌ Don't

`let name1 = "Correct"` ✅ Do

3. Use Meaningful Names

Exp : `let d = 90` ❌ Don't

`let studentGrade = 90` ✅ Do



4. No Spaces or Special Characters

Only letters, digits, _ (underscore), or \$ (dollar sign) allowed.
No hyphens (-), spaces, or symbols

Exp : `let first-name = "quad"`

✗ Don't

`let first_name = "quad"`

✓ Do

5. Avoid Reserved Keywords

You can't use JavaScript reserved words like `let`, `class`, `function`, etc., as variable names.

Exp : `let function = "quad"`

✗ Don't

`let myFunction = "quad"`

✓ Do

Note : JavaScript is case-sensitive language





04

Data Types

Data types in JavaScript

There are two types of Data types :



Primitive

primitive data types are the most basic and **fundamental types of data**. They represent **single values**, are **immutable** (cannot be changed)

Non - Primitive

non-primitive data types (also called **reference types**) are types that **can store multiple values** and are **mutable**.



Primitive Data types

- **Stored by value**, not by reference
- **Immutable** (once created, their value cannot be changed)
- Fast access and efficient memory usage
- **Not objects** (except null, which returns 'object' due to a known bug)

Non-Primitive Data types

- **Stored by reference**, not by value
- **Mutable** (their contents can be changed)
- Can store **collections of data** or **complex entities**

Note :

The variable **does not store the actual data directly**, but instead **stores a reference (memory address)** that points to where the data is located in memory



List of Primitive Data Types in JavaScript:

Number	Exp :	10,1,10.1	Any number (integer or floating-point)
BigInt	Exp :	123456789n	For very large integers
String	Exp :	"quad"	Sequence of characters
Boolean	Exp :	true, false	Logical values
undefined	Exp :	let x;	Declared but not assigned a value
null	Exp :	let y = null;	Represents intentional "no value"
Symbol	Exp :	Symbol('id')	Unique and immutable value (ES6+)



List of Non-Primitive Data Types in JavaScript:

Object	Exp :	{name: "Nitish", age: 25}	Key-value pairs, arrays, functions
Array	Exp :	[1, 2, 3]	Ordered collection of items
Function	Exp :	function greet() {}	A block of reusable code

Note :

The typeof operator is used to **check the data type** of a given value in JavaScript

Exp :

```
Let myName ="quad"
```

```
Console.log(typeof myName)
```

```
// output : string
```

Exp :

```
Let x=1
```

```
Console.log(typeof x)
```

```
// output : Number
```

